

Lossless vs Lossy Decomposition

Core concepts, step-by-step examples, spurious tuples, and binary lossless-join tests

★ **Golden Rule:** Lossless decomposition guarantees that natural joining the decomposed relations yields exactly the original relation—no lost information and no extra spurious tuples.

1. Core Definitions

1. Lossless Decomposition

Reconstructing the original relation R from decomposed relations R_1 and R_2 using natural join ($R_1 \bowtie R_2$) yields **exactly** the original dataset.

Formula: $R_1 \bowtie R_2 = R$

2. Lossy Decomposition

Joining the decomposed relations produces extra, invalid tuples called **spurious tuples**, leading to inconsistent or incorrect query results.

Formula: $R_1 \bowtie R_2 \supset R$

⚠ Common Misconception

"Lossy" does **NOT** mean rows are lost or missing. It means the decomposition causes a loss of structural integrity/information by adding *extra, incorrect* tuples that distort original facts.

2. Detailed Example: Lossy Decomposition

Consider relation $R(A, B, C)$ decomposed into $R_1(A, B)$ and $R_2(B, C)$:

Original $R(A, B, C)$			Decomposed $R_1(A, B)$		Decomposed $R_2(B, C)$	
A	B	C	A	B	B	C
1	2	1	1	2	2	1
2	2	2	2	2	2	2
3	3	2	3	3	3	2

Natural Join Result ($R_1 \bowtie R_2$ on Common Attribute B):

Joining on common attribute **B** produces 5 tuples instead of the original 3:

A	B	C	Status
1	2	1	✓ Original Tuple
1	2	2	✗ Spurious Tuple
2	2	1	✗ Spurious Tuple
2	2	2	✓ Original Tuple
3	3	2	✓ Original Tuple

Why did this happen? Attribute **B** contains repeating values (2, 2, 3) and cannot uniquely identify tuples. When joining on **B = 2**, 2 rows from R_1 match 2 rows from R_2 , generating $2 \times 2 = 4$ combinations (including 2 spurious tuples).

3. The Binary Lossless-Join Condition ★★☆☆

For a binary decomposition of relation R into R_1 and R_2 , the decomposition is **Lossless** if and only if the common attribute set is a **superkey** of at least one decomposed relation:

Binary Lossless Condition:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{OR} \quad (R_1 \cap R_2) \rightarrow R_2$$

Testing the Conditions on Examples:

- **Testing Lossy Example:** Common attribute $R_1 \cap R_2 = \{B\}$.
Is $B \rightarrow A$? **No**. Is $B \rightarrow C$? **No**.
→ B is not a superkey for R_1 or R_2 , making the decomposition **Lossy** ❌.
- **Testing Lossless Example:** $R(A, B, C)$ decomposed into $R_1(A, B)$ and $R_2(A, C)$ with FD $A \rightarrow B$.
Common attribute $R_1 \cap R_2 = \{A\}$. Since $A \rightarrow B$, A determines all attributes of R_1 ($A \rightarrow AB$).
→ A is a superkey for R_1 , making the decomposition **Lossless** ✅.

4. Three Essential Rules for Valid Decomposition

1. **Attribute Preservation:** The union of decomposed attributes must cover all original attributes ($R_1 \cup R_2 = R$).
2. **Common Attribute:** The decomposed relations must share at least one attribute ($R_1 \cap R_2 \neq \emptyset$).
3. **Superkey Condition:** The common attribute(s) must form a superkey of at least one of the decomposed relations.

5. Comparison: Lossless vs Lossy Decomposition

Feature	Lossless Decomposition	Lossy Decomposition
Join Result ($R_1 \bowtie R_2$)	Exactly original relation R	Original relation + Spurious tuples
Information Integrity	✓ Fully Recoverable	✗ Distorted / Incorrect Results
Spurious Tuples	No	Yes
Database Consistency	Maintained perfectly	Causes inconsistency & false data
Desired in Normalization?	✓ Yes (Mandatory)	✗ No (Forbidden)

6. Connection to Normalization (3NF vs BCNF)

Lossless decomposition is required across all normal forms ($1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$), but it must not be confused with **Dependency Preservation**:

- **3NF Decomposition:** Standard 3NF synthesis guarantees both **Lossless Join** AND **Dependency Preservation**.
- **BCNF Decomposition:** BCNF guarantees **Lossless Join**, but **Dependency Preservation** is **NOT** always guaranteed.

🧠 Quick Revision & Memory Trick

Lossless Rule: For binary decomposition $R \rightarrow R_1, R_2$ with $X = R_1 \cap R_2$, check if $X \rightarrow R_1$ OR $X \rightarrow R_2$. If true → Lossless; if false → Lossy.

One-Line Memory Trick: "The common attribute must be powerful enough to determine at least one whole table."